
KHAMSAT PREDICTOR

Ismael Mousa

An-Najah National University
ismaelramzimousa@gmail.com

ABSTRACT

Presents the Khamsat Predictor, a machine learning model developed to address pricing challenges in the freelance marketplace. By leveraging data from Khamsat [1], the largest Arabic freelance platform, the model provides accurate and data-driven price predictions based on service characteristics. The core objective is to enhance transparency and trust, alleviating price-related uncertainty for both sellers and clients. Focuses on the unique challenges posed by the dataset, including its small size and class imbalance, and explore the impact of these factors on model performance. Experimental results, obtained from both imbalanced and balanced datasets, demonstrate the model's effectiveness in delivering reliable price estimates. The proposed approach integrates classical machine learning techniques, from data collection to model deployment, ensuring a robust and scalable solution for real-world applications.

1 Introduction

In the freelance marketplace, two persistent challenges significantly impact both sellers and clients: trust and price uncertainty. Sellers often face difficulty in determining optimal pricing, with concerns about either overpricing, which could drive clients away, or underpricing, which undermines the value of their work. On the other hand, clients worry about paying excessively for services that may not meet their expectations. To address these challenges, a solution is needed that fosters fairness, transparency, and trust within the marketplace.

The Khamsat Predictor aims to provide such a solution by leveraging data-driven insights. The model offers accurate price predictions based on a range of service characteristics extracted from the platform. Built using classical machine learning techniques, the model strives to offer fair and reliable pricing recommendations. By improving transparency and reducing price-related anxiety, this approach seeks to enhance user confidence and promote efficiency within the freelance ecosystem.

2 Model Architecture

The Khamsat Predictor adopts a multimodal approach, evaluating and optimizing several machine learning algorithms through hyperparameter tuning. Key models include SoftMax Regression, Support Vector Classifier (SVC), and Random Forest Classifier [2, 3, 4]. The mathematical formulations and implementations of these models are based on Scikit-learn [5], rather than being manually coded.

3 Training

This section describes the training regime for our models.

3.1 Data Splitting

The dataset used for training and evaluation consists of service listings from Khamsat. Initially, the models are trained on the clean, imbalanced dataset, where the target variable, *Price*, exhibits class imbalance. The dataset is split into training, validation, and test sets, with stratification applied to ensure balanced class distribution in each subset. Specifically, 10% of the data is reserved for testing, and 10% of the training data is further split for validation, which is used for hyperparameter tuning.

Subsequently, I address the class imbalance by upsampling the minority class in the training set to achieve a balanced distribution. The models are retrained on this upsampled dataset, ensuring that the training data is balanced while maintaining the same evaluation split (with 10% reserved for testing and 10% of the training data for validation).

3.2 Hyperparameter Tuning

Optuna [6] is used for hyperparameter optimization, conducting up to 50 trials for each model to maximize the F1 score [7]. Tables 1 and 2 summarize the hyperparameters tuned for each model.

Table 1: Hyperparameters tuned on the imbalanced dataset.

Hyperparameter	SoftMax Regression	SVC	Random Forest Classifier
Class Weight	balanced	balanced	balanced
Multi Class	multinomial	-	-
Solver	lbfgs	-	-
Kernel	-	linear	-
Gamma	-	0.48	-
C	0.53	0.79	-
Max Iterations	6972	8170	-
Decision Function	-	ovo	-
Estimators	-	-	395
Depth	-	-	14
Criterion	-	-	log loss

Table 2: Hyperparameters tuned on the balanced dataset.

Hyperparameter	SoftMax Regression	SVC	Random Forest Classifier
Class Weight	balanced	balanced	balanced
Multi Class	multinomial	-	-
Solver	lbfgs	-	-
Kernel	-	rbf	-
Gamma	-	0.33	-
C	0.88	0.57	-
Max Iterations	7356	9189	-
Decision Function	-	ovo	-
Estimators	-	-	133
Depth	-	-	22
Criterion	-	-	log loss

3.3 Evaluation Metrics

The performance of the models is assessed using a set of standard evaluation metrics, including Log Loss [8], Accuracy [9], Precision [10], Recall [11], and the F1-score. These metrics provide a comprehensive view of the model's ability to generalize and its effectiveness in handling both balanced and imbalanced datasets. Log loss is particularly useful for evaluating probabilistic models, while accuracy gives an overall assessment of the correct predictions. Precision, Recall, and F1-score are employed to understand the model's performance in terms of positive class identification, especially in cases of class imbalance.

The results for each model, presented in Tables 3 and 4, highlight the trade-offs between these metrics under different conditions, offering insight into the strengths and weaknesses of each approach.

Table 3: Performance metrics on the imbalanced dataset.

Model	Loss	Accuracy	Precision	Recall	F1
SoftMax Regression	1.62	40%	17%	23%	17%
SVC	1.18	42%	16%	18%	16%
Random Forest Classifier	1.28	56%	22%	25%	22%

Table 4: Performance metrics on the balanced dataset.

Model	Loss	Accuracy	Precision	Recall	F1
SoftMax Regression	0.95	68%	66%	68%	67%
SVC	0.05	98%	98%	98%	98%
Random Forest Classifier	0.23	97%	97%	97%	97%

3.4 Implementation Details

The implementation follows a structured workflow designed to maximize the effectiveness of the machine learning model by focusing on an in-depth understanding of the dataset collected from Khamsat. The workflow consists of distinct stages, each functioning as a standalone unit: data exploration and analysis, preprocessing, and modeling.

Data Exploration and Feature Selection: Over three to four weeks of data analysis, I examined the dataset to understand its structure, distributions, and relationships between features. Based on these insights, I selected the most impactful features for training the model and devised appropriate encoding techniques to handle various data types effectively.

Feature Engineering and Encoding: Several manual and automated feature engineering steps were undertaken to prepare the data:

- **Duration Mapping:** Categorical values in the *Duration* feature were manually mapped to integers representing the corresponding number of days. The mapping values were stored in JSON files for modularity and ease of reuse.
- **Response Time Conversion:** Text-based response times (e.g., "minutes" to "days") were standardized into hours. A utility was developed to automate this conversion and round values to two decimal points for consistency. The mappings were also stored in JSON file under their respective feature names.
- **Owner Level Encoding:** Recognizing the significant impact of the *Owner Level* feature, a custom manual encoding was implemented. Each level was assigned a value reflecting its pricing patterns, from 5 for beginner levels to 50 for advanced levels. This encoding technique, based on human-driven hypotheses, outperformed traditional methods such as ordinal and label encoding.

- **Category and Service Name Encoding:** For categorical features like *Category Name* and *Service Name*, a customized one-hot encoding method was employed to ensure compatibility with the deployment stage.

Preprocessing Decisions: Scaling techniques were deliberately avoided, as analysis and trials indicated no notable improvement in performance. Placeholder values in certain features were addressed using manual mappings and custom utilities, ensuring clean and standardized input for the model.

Modeling and Integration: The modeling process was isolated as a separate unit to maintain a clear and iterative workflow. The focus remained on ensuring that each stage, data analysis, preprocessing, and modeling, was well defined and reproducible.

This structured approach, emphasizing meticulous feature engineering and modular preprocessing, laid the foundation for an efficient and scalable implementation of the Khamsat Predictor.

4 Experiments

The experiments conducted in this study were aimed at evaluating model performance on both the original imbalanced dataset and a balanced version created through upsampling. Given the relatively small size of the dataset (7,818 rows) and its inherent class imbalance, careful attention was paid to understanding how these factors affected model accuracy and generalization.

Initially, models were trained on the imbalanced dataset, which posed significant challenges due to the skewed distribution of target values. To address this, a series of experiments were also performed using a balanced dataset, obtained through upsampling techniques. This allowed for a more equitable representation of target classes, providing a clearer assessment of model performance across all classes.

Both experimental setups were tracked using MLflow [12], ensuring reproducibility and optimization of the model workflows. The impact of the dataset imbalance and the effectiveness of upsampling on model performance were evaluated across various metrics, providing insights into the trade-offs between Accuracy, Precision, Recall, and F1-score under different data conditions.

4.1 Computation

The training process for the Khamsat Predictor was executed entirely on CPU-based systems due to computational constraints and the reliance on classical machine learning techniques. Specifically, the implementation leveraged Python’s data science stack, including Pandas [13], NumPy [14], and Scikit-learn, without utilizing GPU acceleration. This approach aligned with the project’s permitted resources, ensuring accessibility and reproducibility.

Training times varied based on the complexity of each model and the dataset size. For the imbalanced dataset, average training times ranged between 2.4 - 37.2 minutes per model. Balancing the dataset through upsampling slightly increased computational costs, with training times extending to 15.6 minutes – 6.44 hours per model. The total training time amounted to 12.32 hours.

Key computational resources included:

- **Processor:** 11th Gen Intel® Core™ i7-1165G7 @ 2.80GHz × 8.
- **Memory:** 16 GB DDR4 RAM.

Despite the absence of GPU acceleration, Scikit-learn’s computational efficiency ensured feasible training times, even during hyperparameter optimization. For instance, Random Forest Classifier required the most time due to its ensemble nature. Hyperparameter tuning using Optuna with 50 trials per model accounted for the majority of computational overhead.

While GPU acceleration could expedite hyperparameter searches and training, its integration would require significant changes beyond the permitted resource scope, such as adopting GPU-compatible libraries like TensorFlow or PyTorch. This was deemed unnecessary for the project's focus on classical machine learning.

4.2 Datasets

The dataset used in this study consists of 7,818 entries from various service categories on the Khamsat platform, with 26 features encompassing both numerical and categorical attributes. The dataset is exclusively in Arabic and is free from missing or duplicated entries in the conventional sense. However, approximately 15.41% of the data contains placeholder values that were addressed during preprocessing. Key features include service details (e.g., Service Name, Offer Stars, Additions Price) and seller characteristics (e.g., Owner Level, Owner Verified).

Initial exploratory analysis revealed several challenges, including biases in feature distributions and inconsistencies in data formatting. For instance, certain numerical fields, such as prices and response times, were stored as text, necessitating conversion for analysis. Moreover, the data exhibited notable biases, such as overrepresentation of low-price services and specific seller levels.

To address the placeholder values, the K-Nearest Neighbors [17] algorithm was employed for imputation. Alternative strategies, such as using statistical measures (e.g., median) or dropping rows with placeholders, were tested but found suboptimal. Median imputation led to poorer model performance, while dropping affected the dataset's balance, especially for underrepresented target classes, such as the \$45 price point, further exacerbating class imbalance.

Additionally, to counteract imbalances in the dataset, upsampling techniques were applied. These adjustments ensured a more equitable representation of all target classes, improving the dataset's suitability for training and evaluation.

4.3 Results

The study achieved notable progress in preprocessing, including the cleaning of text-based numerical features and the identification of significant correlations between key variables. Preliminary evaluations demonstrate the model's strong ability to predict prices with high accuracy, highlighting its potential for practical applications in the freelance marketplace.

5 Conclusion

In this work, I presented the Khamsat Predictor, a machine learning model designed to address pricing challenges in the freelance marketplace. By leveraging data-driven insights, our model replaces subjective price estimation with transparent and reliable recommendations, fostering greater trust between sellers and clients.

For price prediction tasks, the Khamsat Predictor demonstrates significant potential in improving market efficiency and fairness. Through careful preprocessing and model tuning, our approach achieves robust performance, providing actionable insights for real-world applications in freelancing.

I am excited about the broader implications of machine learning in enhancing marketplace dynamics and plan to extend our approach to other freelance platforms and pricing models. Future directions include integrating additional features, exploring multilingual datasets, and adapting the model to handle more dynamic or domain-specific pricing scenarios.

The code used to train and evaluate our models is available at <https://github.com/IsmaelMousa/khamsat-predictor>.

Acknowledgments

I am grateful to Prof. Adnan Salman and Eng. Samer Huwari for their fruitful comments, corrections and inspiration.

References

- [1] Khamsat Platform.
<https://khamsat.com>.
- [2] SoftMax Regression. Wikipedia.
https://en.wikipedia.org/wiki/Multinomial_logistic_regression.
- [3] Support Vector Classification (SVC). Wikipedia.
https://en.wikipedia.org/wiki/Support_vector_machine.
- [4] Random Forest Classifier. Wikipedia.
https://en.wikipedia.org/wiki/Random_forest.
- [5] Scikit-learn Documentation.
<https://scikit-learn.org>.
- [6] Optuna Documentation.
<https://optuna.org>.
- [7] F1-score. Wikipedia.
<https://en.wikipedia.org/wiki/F-score>.
- [8] Cross-entropy loss. Wikipedia.
<https://en.wikipedia.org/wiki/Cross-entropy>.
- [9] Accuracy score. Wikipedia.
https://en.wikipedia.org/wiki/Accuracy_and_precision.
- [10] Precision metrics. Wikipedia.
https://en.wikipedia.org/wiki/Precision_and_recall.
- [11] Recall metric. Wikipedia.
https://en.wikipedia.org/wiki/Precision_and_recall.
- [12] MLflow Documentation.
<https://mlflow.org>.
- [13] Pandas Documentation.
<https://pandas.pydata.org>.
- [14] NumPy Documentation.
<https://numpy.org>.
- [15] Selenium Documentation.
<https://www.selenium.dev>.
- [16] FastAPI Documentation.
<https://fastapi.tiangolo.com>.
- [17] K-Nearest Neighbors Algorithm. Wikipedia.
https://en.wikipedia.org/wiki/K-nearest_neighbors_algorithm.